

Seller and Inventory Management System

Scenario: An e-commerce enterprise handles merchandise sourced from a variety of suppliers. The database architecture must store supplier details, track merchandise provided by each supplier, and oversee stock levels to maintain supply readiness.

1. Create Seller and Inventory Tables:

- Form the Seller table structure with attributes for vendor identifier, vendor name, email address, contact phone, and location address.
- Form the Inventory table structure with attributes for stock identifier, item identifier, vendor identifier, stock level, availability condition, and update date.

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7), Seller_Name VARCHAR(25), Email VARCHAR(30), Phone_Number NUMBER(10), Address VARCHAR(45));
Table created.
SQL> CREATE TABLE Inventory (Inventory_ID VARCHAR(7), Product_ID VARCHAR(7), Seller_ID VARCHAR(7), Stock_Quantity NUMBER, Stock_Status VARCHAR(11), Last_Update DATE);
Table created.
```

2. Establish Relationships:

Seller and Product:

- An individual supplier can provide multiple items.
- An individual item can be provided by single or multiple suppliers. **Relationship: Seller → Product operates as one-to-many.**

Product and Inventory:

- Every item possesses corresponding stock records.
- Stock records track current item availability.
Relationship: Product → Inventory operates as one-to-one.

Primary Key added for Both Tables:

- Assign Primary Key constraint to the Seller table on the vendor identifier attribute.
- Assign Primary Key constraint to the Inventory table on the stock identifier attribute.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);
Table altered.
SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY (Inventory_ID);
Table altered.
```

Foreign Key for Inventory Table:

- Configure Foreign Key constraint on the Inventory table pointing to the item identifier in the merchandise table.
- Configure Foreign Key constraint on the Inventory table pointing to the vendor identifier in the seller table.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY (Product_ID) REFERENCES Product (Product_ID);
Table altered.
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);
Table altered.
```

Constraints to make Unique Email ID:

- Enforce Unique constraint on the vendor email field.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE (Email);
Table altered.
```

To make Email NOT NULL:

- Apply NOT NULL condition to the vendor email field.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;
Table altered.
```

Add Constraint to Check the Stock Quantity is Positive:

- Set Check constraint to ensure the stock level remains non-negative (>0).

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity >= 0);
Table altered.
```

Insert Values to Seller Table:

Enter supplier entries interactively using substitution parameters.

```
SQL> INSERT INTO seller VALUES ('SEL001','Nature Hub','naturehub@gmail.com',9876543210,'Chennai');
1 row created.

SQL> INSERT INTO seller VALUES ('SEL002','Eco Living Store','ecoliving@gmail.com',9876543211,'Coimbatore');
1 row created.

SQL> INSERT INTO seller VALUES ('SEL003','Bamboo House','bamboohouse@gmail.com',9876543212,'Bangalore');
1 row created.

SQL> INSERT INTO seller VALUES ('SEL004','Green Basket','greenbasket@gmail.com',9876543213,'Madurai');
1 row created.

SQL> INSERT INTO seller VALUES ('SEL005','Earth Craft','earthcraft@gmail.com',9876543214,'Chennai');
1 row created.
```

```
SQL> SELECT * FROM SELLER;
```

SELLER_	SELLER_NAME	EMAIL	PHONE_NUMBER	ADDRESS
SEL001	Nature Hub	naturehub@gmail.com	9876543210	Chennai
SEL002	Eco Living Store	ecoliving@gmail.com	9876543211	Coimbatore
SEL003	Bamboo House	bamboohouse@gmail.com	9876543212	Bangalore
SEL004	Green Basket	greenbasket@gmail.com	9876543213	Madurai
SEL005	Earth Craft	earthcraft@gmail.com	9876543214	Chennai

Insert Values to Inventory Table:

Enter stock allocation entries linking suppliers, items, quantities, statuses, and dates.

```
SQL> INSERT INTO inventory VALUES ('INV001','PR0001','SEL001',45,'Available',DATE '2026-09-01');
1 row created.

SQL> INSERT INTO inventory VALUES ('INV002','PR0002','SEL007',35,'Available',DATE '2026-09-02');
1 row created.

SQL> INSERT INTO inventory VALUES ('INV003','PR0003','SEL002',40,'Available',DATE '2026-09-03');
1 row created.

SQL> INSERT INTO inventory VALUES ('INV004','PR0004','SEL003',90,'Available',DATE '2026-09-04');
1 row created.
```

```
SQL> SELECT * FROM Inventory;
```

INVENTO	PRODUCT_ID	SELLER_	STOCK_QUANTITY	STOACK_STAT	LAST_UPDA
INV001	PR0001	SEL001	45	Available	01-SEP-26
INV002	PR0002	SEL007	35	Available	02-SEP-26
INV003	PR0003	SEL002	40	Available	03-SEP-26
INV004	PR0004	SEL003	90	Available	04-SEP-26

1. Seller-wise Product Report

Display:

- Vendor Name
- Quantity of items provided
- Total active stock

Use Joins to get details from Seller and Inventory Table.

```
SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Eco Style India	1	25
Nature Hub	1	45
Green Basket	1	55
Earth Craft	2	135
Bamboo House	2	120
Organic Valley	2	130
Eco Living Store	1	40

7 rows selected.

2. Stock Availability Report

Display:

- Item Name
- Stock Quantity
- Availability Status

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Organic Oats	45	Available
Natural Honey	35	Available
Aloe Face Cream	40	Available
Bamboo Comb	90	Available
Bamboo Tray	30	Available
Steel Lunch Box	55	Available
Eco Diary	70	Available
Cotton Tote Bag	65	Available
Eco Cotton Kurta	25	Available
Organic Seed Pack	95	Available

10 rows selected.

3: Inventory Analysis

Find:

- Total count of available items

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_Status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
10

- Total count of out-of-stock items

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Status = 'Out of Stock';
```

OUTOFSTOCK_PRODUCTS
0

- Items holding peak stock quantities

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);
```

PRODUCT_NAME	STOCK_QUANTITY
Organic Seed Pack	95

- Mean stock level across all items

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;
```

AVERAGE_INVENTORY
55